



Exercises session 2

Decision problems, multitape turing machines, non-deterministic turing machines

1 Definitions

Definition 1.1 (Languages and decision problems) *We will now define languages and their corresponding decision problems*

- *Alphabet Σ : a finite set of symbols.*
- *Word w : a finite sequence of symbols from Σ .*
- *Σ^* : The set of all words over the alphabet Σ .*
- *Language L : a set of words.*

We define the decision problem associated to a language L as follows: Given $w \in \Sigma^$, decide if $w \in L$ (or $w \notin L$).*

Definition 1.2 (Multitape Turing Machine) *A deterministic TM with k tapes is defined by $(Q, \Sigma, \Gamma, \delta, q_0, q_{\text{accept}}, q_{\text{reject}})$, where*

- *Q is a finite set of states*
- *Σ is the input alphabet, with $\vdash, B \notin \Sigma$*
- *Γ is the tape alphabet, with $\Sigma \cup \{\vdash, B\} \subseteq \Gamma$*
- *$q_0 \in Q$ is the initial state*
- *$q_{\text{accept}}, q_{\text{reject}} \in Q$ are the accepting and rejecting state*
- *δ is a partial function:*

$$\delta : Q \times \Gamma^k \rightarrow Q \times \Gamma^k \times \{\leftarrow, \square, \rightarrow\}^k.$$

δ is called the transition function.

Definition 1.3 (Non-deterministic Turing Machine) *A non-deterministic turing machine is defined as $(Q, \Sigma, \Gamma, \delta, q_0, q_{\text{accept}}, q_{\text{reject}})$ where*

- *Q is a finite set of states*
- *Σ is the input alphabet, with $\vdash, B \notin \Sigma$*

- Γ is the tape alphabet, with $\Sigma \cup \{\vdash, B\} \subseteq \Gamma$
- $q_0 \in Q$ is the initial state
- $q_{accept}, q_{reject} \in Q$ are the accepting and rejecting states
- δ is a **relation**:

$$\delta \subseteq Q \times \Gamma \times Q \times \Gamma \times \{\leftarrow, \square, \rightarrow\}$$

2 Session exercises

1. Reformulate the following problems into a decision problem
 - (a) Given a graph, and two special nodes s and t , you want to find a path from s to t
 - (b) Given a graph, a node s , and a set of nodes V , what is the shortest possible path that visits all the nodes in V
 - (c) You're organising a dinner party. Although you want to invite all of your friends, some of them can't stand each other, and you want a conflict-free dinner.
2. Describe informally (but clearly) a multi-tape turing machine for each of the following problems.
 - $L_1 = \{w : w \text{ has the same amount of 0s and 1s}\}$
 - $L_2 = \{a^n b^n c^n : n \in \mathbb{N}\}$
 - $L_3 = \{w \cdot w^{-1} : w \in \{0, 1\}^*\}$
3. Let's introduce (recall?) boolean formulas
 - **Syntax of boolean formulas:** let's consider a set of variables **Vars**: $\{x_1, \dots, x_n\}$ and operators $\vee, \wedge, \neg, \rightarrow, \leftrightarrow$, then we define boolean formulas recursively as
 - every variable x_i is a boolean formula
 - if φ and ϕ are boolean formulas, then $(\neg\varphi)$, $(\varphi \wedge \phi)$, $(\varphi \vee \phi)$, $(\varphi \rightarrow \phi)$ and $(\varphi \leftrightarrow \phi)$ are also boolean formulas
 - **Semantics of boolean formulas:** a truth assignment σ maps each variable x_i to either **true** (1) or **false** (0), formally $\sigma : \mathbf{Vars} \rightarrow \{0, 1\}$. We can evaluate a boolean formula ϕ using a truth assignment σ : we will denote this as $\phi(\sigma)$
 - the evaluation of a variable x_i corresponds to $\sigma(x_i)$
 - if ϕ_1 and ϕ_2 are boolean formulas, then if :
 - * $\varphi = (\neg\phi)$, then $\varphi(\sigma) = 1 - \phi(\sigma)$
 - * $\varphi = (\phi_1 \wedge \phi_2)$ then $\varphi(\sigma) = \min\{\phi_1(\sigma), \phi_2(\sigma)\}$

- * $\varphi = (\phi_1 \vee \phi_2)$ then $\varphi(\sigma) = \max\{\phi_1(\sigma), \phi_2(\sigma)\}$
- * $\varphi = (\phi_1 \rightarrow \phi_2)$ then $\varphi(\sigma) = 1$ iff $\phi_1(\sigma) = 0$ or $\phi_2(\sigma) = 1$
- * $\varphi = (\phi_1 \leftrightarrow \phi_2)$ then $\varphi(\sigma) = 1$ iff $\phi_1(\sigma) = \phi_2(\sigma)$
- **Satisfiability:** we say a boolean formula φ is satisfiable, if there exists a truth assignment σ such that $\varphi(\sigma) = 1$
- **Conjunctive normal form:** we say a boolean formula is in conjunctive normal form (or CNF) if its a conjunction of disjunctions of literals. A literal is a variable or a negation of a variable. An example:

$$\varphi = (x_1 \vee \neg x_2) \wedge (x_1 \vee x_3)$$

In this exercise we will introduce the SAT problem, that will play an important role in future lessons of this course.

- (a) Given a CNF boolean formula φ and an assignment σ for such formula, give a TM that *verifies* whether $\varphi(\sigma) = 1$. What is the running time of such algorithm?
- (b) Now lets consider the following decision problem

$$\text{SAT} = \{\varphi : \varphi \text{ is a satisfiable CNF boolean formula} \}$$

Can we use the idea of the question above to solve this? What would be the running time?

- (c) What if we try using non-deterministic turing machines?

Solutions

1. Reformulate the following problems into a decision problem

- (a) Given a graph, and two special nodes s and t , you want to find a path from s to t
- (b) Given a graph, a node s , and a set of nodes V , what is the shortest possible path that visits all the nodes in V
- (c) You're organising a dinner party. Although you want to invite all of your friends, some of them can't stand each other, and you want a conflict-free dinner.

(a) the input is a graph $G=(V, E)$ and nodes $s, t \in V$
"Finding a path" is not a decision problem as it's not a YES/NO problem. But we can do

"is there a path ..."

$$L = \{ (G, (s, t)) : \text{there's a path } \pi = v_0 \dots v_k \text{ in } G \text{ such that } v_0 = s, v_k = t \}$$

(b) we should probably also define path: a path in a graph is a sequence of nodes $v_0 \dots v_k$, each $v_i \in V$ and every $(v_i, v_{i+1}) \in E$.

(b) Similarly to (a), we cannot state "finding a path" as a decision problem.

In this case, on top of that, we're worried about the length of the path.

What we can do is to choose a " k " and limit the length of paths we're interested in

$$G = (V, E) \quad V \subseteq \mathcal{V}$$

$$L = \{ (G, V, k) : \text{there's a path } \pi = v_1 \dots v_m \text{ st } \forall v \in V, \exists v = v_i \in \pi, \text{ and } m \leq k \}$$

(c) we can code this as a graph $G=(V, E)$ where V is a set of all your friends (coded as vertices) and we include $(v_i, v_j) \in E$ if v_i and v_j hate each other

Then we state the problem as finding a set of vertices that are not connected to each other. We're worried about the size of such set

$$L = \left\{ G=(V,E), k : \exists V \subseteq V : |V| \geq k \text{ and } \forall (u,v) \in V \times V, (u,v) \notin E \right\}$$

2. Describe informally (but clearly) a multi-tape turing machine for each of the following problems.

- $L_1 = \{w : w \text{ has the same amount of 0s and 1s}\}$
- $L_2 = \{a^n b^n c^n : n \in \mathbb{N}\}$
- $L_3 = \{w \cdot w^{-1} : w \in \{0,1\}^*\}$

(L₁) we can do this with 2-tapes, first for scanning the input, erasing all the 1's, and writing them in the 2nd tape. after we can go matching.

(L₂) we can do this with 3 tapes, first scan and check the "structure" then on the way back write the amount of c's on both tapes 2 and 3.
when we encounter first b, we match with the 2nd tape, and when we encounter first a, we match it with the third tape
(also in 2; reusing space)

(L3) We can do this with 3 tapes.

First go all the way to the end of the word, on the way back we write all the letters on tape 2, and then we just match them.

BUT... what if we have an odd word like

$100 \downarrow 00 \downarrow$ $\rightarrow$ this shouldn't be in L.

So we can use the 3rd tape to check this: we can just keep the head on the same spot on the 3rd tape, write a $\downarrow$ and erase it... at the end, if there's a $\downarrow$ in the 3rd tape, reject.

- (a) Given a CNF boolean formula φ and an assignment σ for such formula, give a TM that *verifies* whether $\varphi(\sigma) = 1$. What is the running time of such algorithm?
- (b) Now let's consider the following decision problem

$$\text{SAT} = \{ \varphi : \varphi \text{ is a satisfiable CNF boolean formula} \}$$

Can we use the idea of the question above to solve this? What would be the running time?

- (c) What if we try using non-deterministic turing machines?

(a) we can use as input, the truth assignment of our formula, and it would look like this

$$\varphi = (x_1 \vee x_2) \wedge (x_2 \vee \neg x_3) \wedge (x_1 \vee x_2)$$

$$\sigma : x_1 \rightarrow 0$$

$$x_2 \rightarrow 1$$

$$x_3 \rightarrow 0$$

1	(	0	$\vee$	1	)	$\wedge$	(	x_2	..	..	..
---	---	---	--------	---	---	----------	---	-------	----	----	----

we should first remove negations, and replace

$\neg 1 \rightarrow 0$
 $\neg 0 \rightarrow 1$

if we can just ignore 0's after

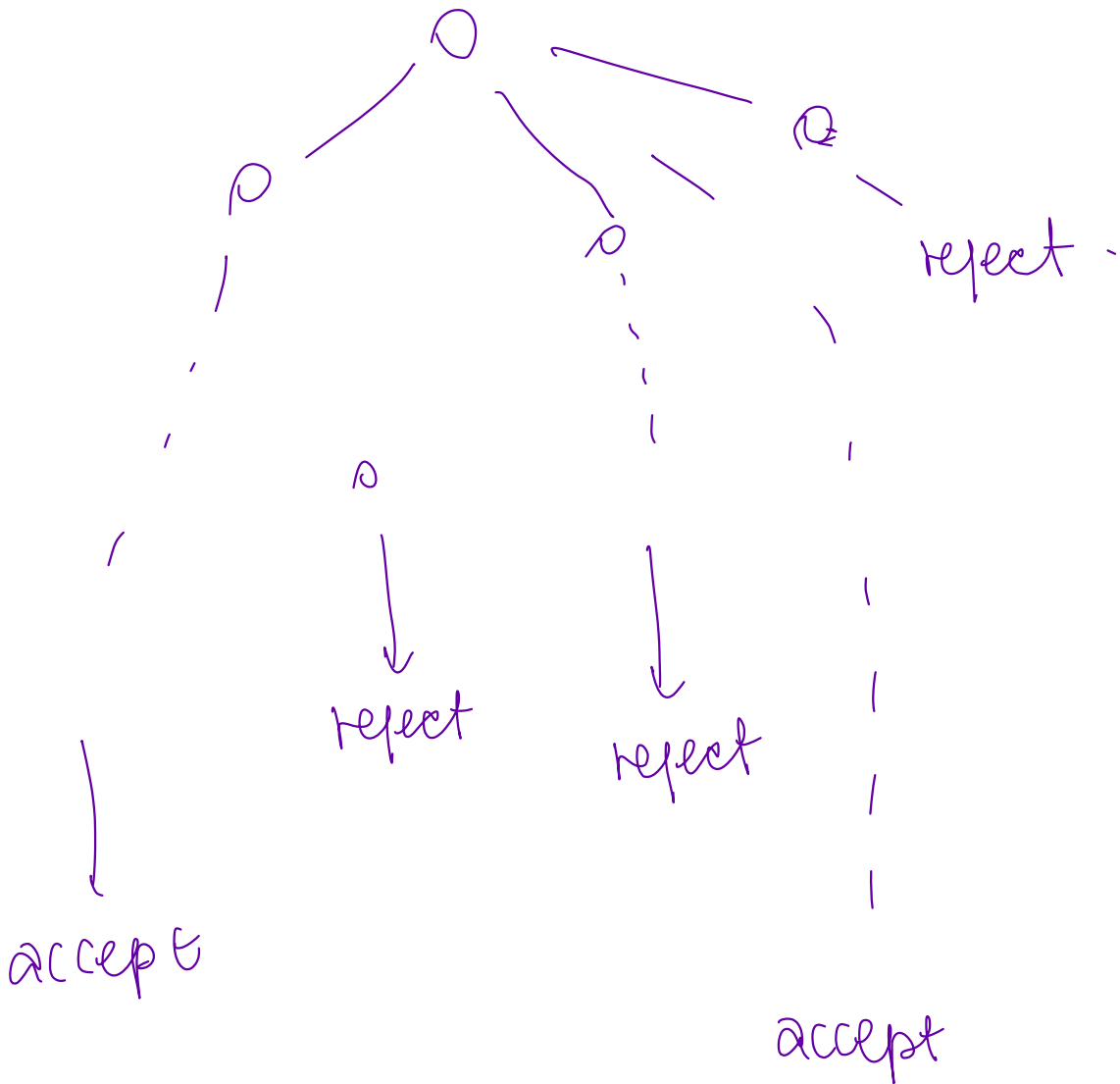
then after we can see if there's at least one 1 between each parenthesis. this should be doable in linear time.

(b) and (c)

Let's make some remarks:

- for a formula φ with variables $x_1, \dots, x_n$ there are 2^n possible assignments
- run above algorithm for every single assignment doesn't seem like a very «good» idea (why?)
- what would be the running time of that?
- deciding whether a particular assignment satisfies a formula seems «easy»
- then deciding if such assignment exists should be easy as well ... or not?

Let's turn our attention into
non-determinism ---



- at each step, we can choose / guess which path to take... how can this help us?